

Capture the Flag 4: The Student List

Goal: Build a scrollable student directory using Dart Lists and `ListView.builder`, replacing manually duplicated widgets with dynamically generated UI.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before building your student list.

Create:

`feature/student-list`

Screenshot:

- Terminal showing your current branch

🚩 Flag 1 — Build Your Student Data

Objective: Store multiple students inside a Dart `List`.

Research what a Dart `List` is and why it is useful when handling multiple pieces of similar data.

Instead of creating one student at a time with separate variables, create a collection that stores at least five students.

Each student should contain information such as:

- Image
- Name
- Course
- Year Level
- Age
- Hobby

Your data should exist separately from your UI widgets.

Screenshot:

Code:

- Your student list
- At least five student entries

```
}

List<Map<String, dynamic>> profiles = [
  {
    'image': 'assets/da.jfif',
    'name': "James Laurence C. Gulfan",
    'courseSection': "BSII - 3",
    'age': 19,
    'hobby': "Basketball",
  },
  {
    'image': 'assets/john.jfif',
    'name': "John Deacon",
    'courseSection': "BSII - 3",
    'age': 23,
    'hobby': null,
  },
  {
    'image': 'assets/brian.jfif',
    'name': "Brian May",
    'courseSection': "BSII - 3",
    'age': null,
    'hobby': "Sleeping",
  },
  {
    'image': 'assets/fred.jfif',
    'name': "Fred Merc",
    'courseSection': "BSII - 3",
    'age': 20,
    'hobby': "Singing",
  },
  {
    'image': 'assets/roger.jfif',
    'name': "Roger Taylor",
    'courseSection': null,
    'age': 26,
    'hobby': "Dancing",
  },
];
```

🚩 Flag 2 — The Copy-Paste Trap

You already know how to create a Student Card.

You created five student cards manually.

You duplicated the same card design until all five students appear.

Stop.

Imagine having:

- 50 students.
- 500 students.
- 5,000 students.

Would copying the same widget over and over still be practical?

This flag is intentionally making you feel the problem before solving it.

Screenshot:

- Your five manually created student cards (do not change your past code yet)

```

const SizedBox(height: 15),
Text(
  profile['name'] ?? "Name: Empty",
  textAlign: TextAlign.center,
  style: const TextStyle(
    fontSize: 22,
    fontWeight: FontWeight.bold,
  ),
),
const SizedBox(height: 10),
Text(
  profile['courseSection'] ?? "Course: Empty",
  textAlign: TextAlign.center,
  style: const TextStyle(
    fontSize: 18,
  ),
),
const SizedBox(height: 8),
Text(
  "AGE: ${profile['age'] ?? "Empty"}",
  textAlign: TextAlign.center,
  style: const TextStyle(
    fontSize: 18,
  ),
),

```

🚩 Flag 3 — Discover **ListView**

Objective: Make your student directory scrollable using **ListView**.

Research Flutter's **ListView**.

Replace your previous layout with a scrolling layout so users can browse through all of your student cards naturally.

Your application should now allow continuous vertical scrolling.

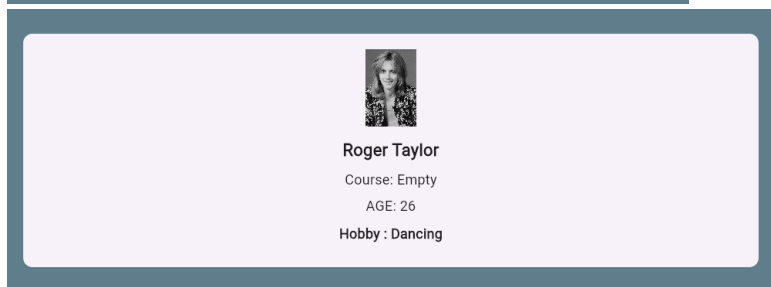
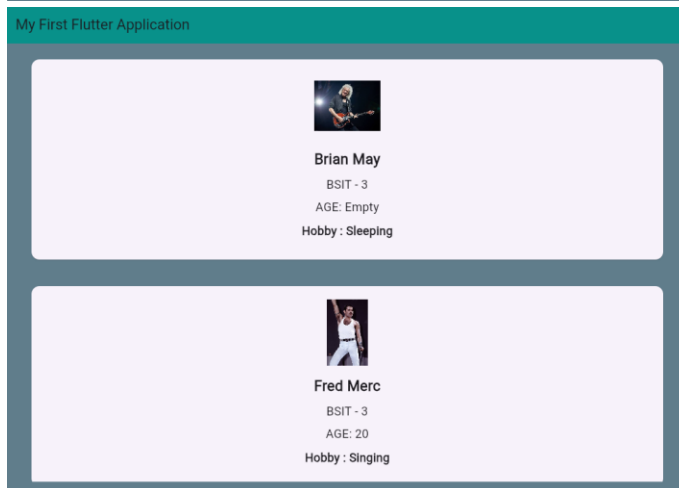
Test

The user should be able to:

- Scroll from the first student
- Continue scrolling
- Reach the last student without resizing the screen

Screenshot:

- `ListView` in your code
- Top of the application
- Bottom portion showing later students



```
child: ListView(  
  children: [  
    ...profiles.map(  
      (profile) => Card(  
        child: Text(  
          profile.name,  
          style: TextStyle(  
            color: Colors.black,  
            fontSize: 16,  
          ),  
        ),  
      ),  
  ],  
),
```

🚩 Flag 4 — Unlock `ListView.builder`

Objective: Generate student cards automatically using `ListView.builder`.

Research `ListView.builder`.

Instead of writing five separate Student Cards, generate them automatically from your student list.

Your UI should now create one card for each student in your list.

Test

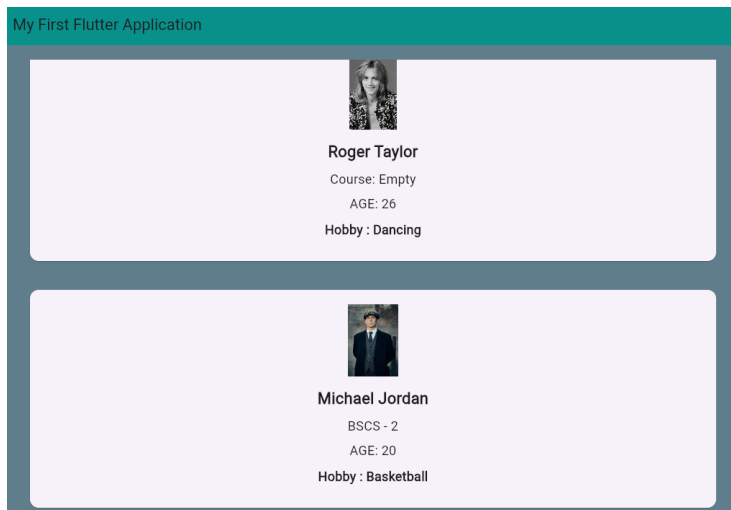
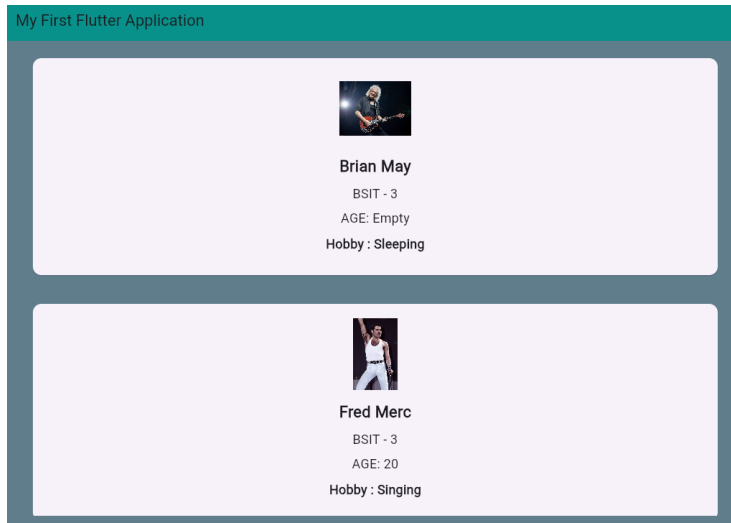
Add one new student to your list.

Without creating another card manually, your application should display the new student automatically.

Screenshot:

- `ListView.builder` in your code
- Student list
- Newly generated student appearing in the app





```
child: ListView.builder(
  itemCount: profiles.length,
  itemBuilder: (context, index) {
    final profile = profiles[index];
```

🚩 Flag 5 — Every Student Deserves an ID

Objective: Expand your student records with additional information.

Real student directories contain more than just names.

Add at least three new fields to every student.

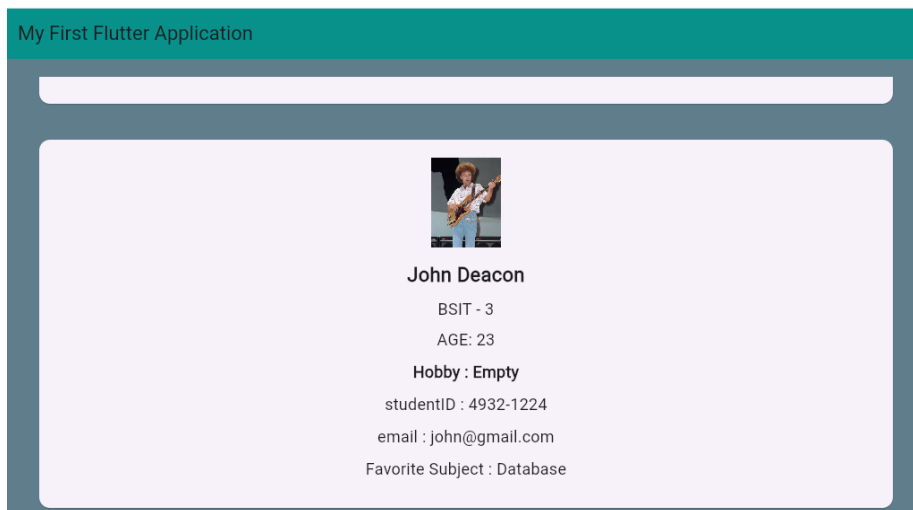
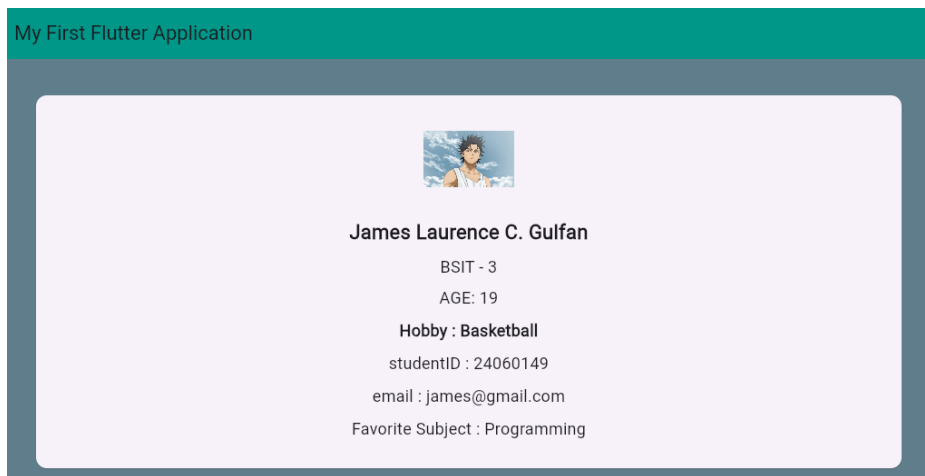
Examples:

- Student ID
- Email
- Favorite Subject

Update your Student Card so the new information appears consistently across every generated card.

Screenshot:

- Updated student data
- Student Card displaying the additional fields



My First Flutter Application



Brian May

BSIT - 3

AGE: Empty

Hobby : Sleeping

studentID : 3003-2044

email : brian@gmail.com

Favorite Subject : Web Development

My First Flutter Application



Fred Merc

BSIT - 3

AGE: 20

Hobby : Singing

studentID : 1302-6953

email : Empty

Favorite Subject : Programming

```

    ),
    const SizedBox(height: 10),
    Text(
      "studentID : ${profile['studentID'] ?? "Empty"}",
      textAlign: TextAlign.center,
      style: const TextStyle(
        fontSize: 18,
      ),
    ),
  ),
  const SizedBox(height: 10),
  Text(
    "email : ${profile['email'] ?? "Empty"}",
    textAlign: TextAlign.center,
    style: const TextStyle(
      fontSize: 18,
    ),
  ),
),
),
List<Map<String, dynamic>> profiles = [
  {
    'image': 'assets/da.jfif',
    'name': "James Laurence C. Gulfan",
    'courseSection': "BSIT - 3",
    'age': 19,
    'hobby': "Basketball",
    'studentID': "24060149",
    'email': "james@gmail.com",
    'favSubject': "Programming",
  },
],

```

Flag 6 — Empty Classroom

What happens when there are no students?

Objective: Handle an empty `List` gracefully.

Imagine your app just connected to a database.

The database returns:

```
[]
```

No students.

What should the user see?

Instead of displaying a broken or confusing screen, research how Flutter can display a different widget when a condition is met.

Create an empty state that clearly communicates that there are currently no students available.

Examples:

- "No students found."
- "Student list is empty."

Test

- Display your student list normally.
- Then temporarily make your list empty.
- Your application should display the empty state instead of student cards.

Screenshot:

- Empty student list
- Empty-state UI
- Application still functioning correctly

No students found.

```
),  
  body: profiles.isEmpty  
    ? const Center(  
      child: Text(  
        "No students found.",  
        textAlign: TextAlign.center,  
        style: TextStyle(  
          fontSize: 22,  
          fontWeight: FontWeight.bold,  
        ),  
      ),  
    ),  
)
```

🚩 Flag 7 — Sort the Roll Call

Objective: Change the order of your student list.

Class lists are rarely displayed randomly.

Research how Dart can rearrange items inside a [List](#).

Sort your students alphabetically by name.

Test

Before sorting:

- Maria
- John
- Aaron

- Sophia

After sorting:

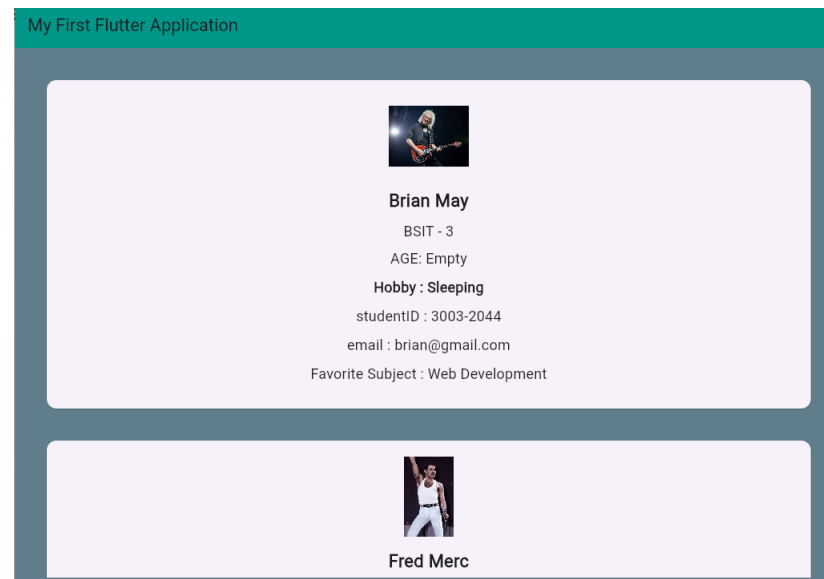
- Aaron
- John
- Maria
- Sophia

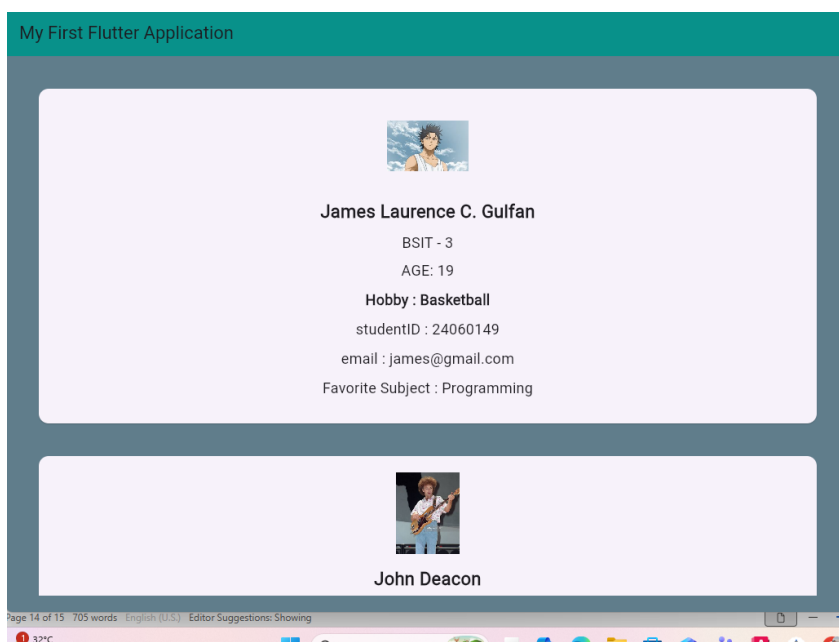
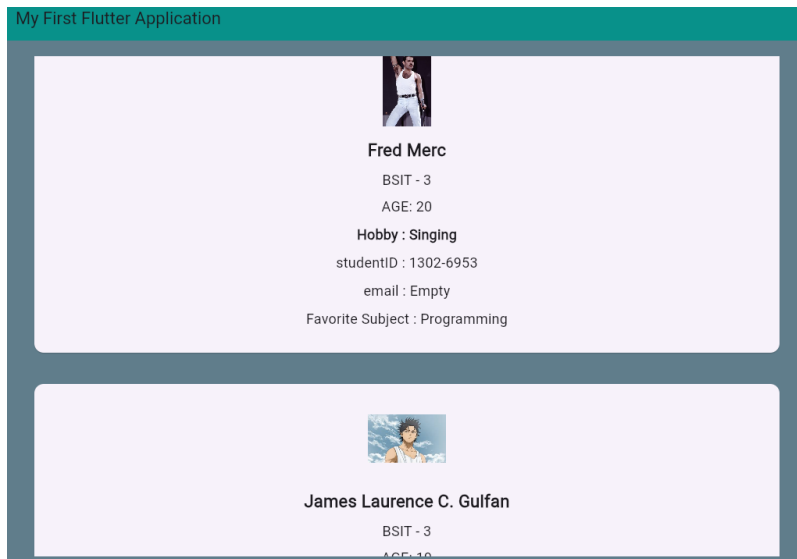
Your UI should immediately reflect the new order.

Screenshot:

- Student list before or after sorting
- Application displaying the sorted order

```
void main() {  
  profiles.sort((a, b) => a['name'].compareTo(b['name']));  
  runApp(const MyApp());  
}
```





My First Flutter Application



John Deacon

BSIT - 3

AGE: 23

Hobby : Empty

studentID : 4932-1224

email : john@gmail.com

Favorite Subject : Database



Roger Taylor

Course: Empty

AGE: 26

My First Flutter Application

Hobby : Dancing

studentID : 2049-232

email : roger@gmail.com

Favorite Subject : Networking



Tommy Shelby

BSCS - 2

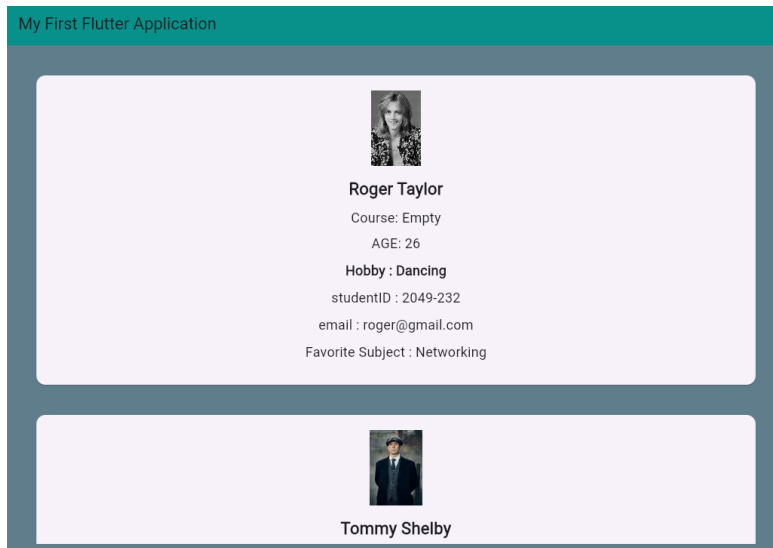
AGE: 20

Hobby : Basketball

studentID : 2309-2060

email : tommy@gmail.com

Favorite Subject : Cybersecurity



🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Student List CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/student-list`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
PS C:\Users\Student\IT314AB_Gulfan\my_first_flutter_app> git commit -m "feat: complete student-list"
[feature/student-list 868be3b] feat: complete student-list
3 files changed, 103 insertions(+), 51 deletions(-)
create mode 100644 my_first_flutter_app/assets/ds.webp
```

🔗 feature/student-list had recent pushes 2 minutes ago

[Compare & pull request](#)

🔗 feature/student-list ▾

🔗 4 Branches

🔗 0 Tags

🔍 Go to file

T

Add file ▾

<> Code ▾

